

队伍编号	BMC2304494
题号	B

基于 NSGA- II 的列车时刻表优化

摘 要

本文针对城市轨道交通列车时刻表优化问题，以企业运营成本最小化和服务水平最大化为目的建立多目标规划模型，利用 NSGA- II 算法对模型进行求解，给出最优的列车开行方案，以此优化并制定列车时刻表，最后基于量化分析结果对铁路运营提出建议。

针对问题一，我们构建了以企业运营成本最小和服务水平最大为目标函数的多目标规划模型。首先，将运行线路分为小交路区间和其他区间，对不同区间进行分类讨论，得到两个区间的企业运营成本和服务水平函数，从而得到整个线路的目标函数。其次，结合实际要求，从停站时间、追踪间隔、客流需求等多方面出发，得到模型的约束条件。最后，利用 NSGA- II 算法得到该模型的最优解集，再使用 Topsis 法从该解集中选出最优解。得到最优的开行方案为：选取 $[S_5, S_{25}]$ 为小交路区间， $[S_1, S_{30}]$ 为大交路区间，开行 13 列大交路列车和 13 列小交路列车，具体结果见文中表 4 “问题一输出”。

针对问题二，我们在问题一制定的列车开行方案下，制定了等间隔的平行运行图。首先，结合断面客流和最小/大停站时间约束条件计算出每个站点的停站时间。其次，结合问题一中已经确定的列车发车频率、列车开行数量和小交路区间即可制定出等间隔的平行运行图。最后，通过 qETRC 软件绘制出等间隔的平行图进行可视化展现，具体输出数据见“附件 7”。

针对问题三，我们基于客流和车站数据，提供量化分析支持，给出了能够降低企业运营成本和提高服务水平的方法或建议。首先，通过问题一建立的目标函数得到企业运营成本和水平与列车最大满载率、列车开行数量、小交路区间、站点位置有关，其次，对以上 4 个影响因素进行灵敏度分析，得到其与企业运营水平和服务水平的量化关系。同时，我们结合实际情况，对问题一给出的模型进行改进，讨论了不同的列车编组可能造成的影响。综上给出能够降低企业运营成本和提高服务水平的方法和建议。

关键词：大小交路 多目标规划模型 NSGA- II 算法 Topsis 法

目录

一、 问题重述	1
1.1 问题背景	1
1.2 问题提出	1
1.2.1 问题一	1
1.2.2 问题二	1
1.2.3 问题三	1
二、 问题分析	1
2.1 问题一的分析	1
2.2 问题二的分析	2
2.3 问题三的分析	2
三、 模型假设与符号说明	2
3.1 模型假设	2
3.2 主要符号说明	3
四、 问题一的模型建立与求解	3
4.1 模型的建立	3
4.1.1 企业运营成本	4
4.1.2 服务水平	5
4.1.3 约束条件	7
4.2 模型的求解算法	7
4.2.1 NSGA-II 算法	8
4.2.2 模型的求解过程及结果	10
五、 问题二的求解	13
5.1 问题二求解过程	13
5.2 问题二求解结果	14
六、 问题三的求解	15
6.1 数据分析	15
6.1.1 站点位置分析	15

6.1.2 满载率灵敏度分析	18
6.1.3 列车开行数量灵敏度分析	19
6.1.4 小交路区间灵敏度分析	19
6.1.5 车辆灵活编组分析	20
6.2 方法和建议	21
6.2.1 降低企业运营成本	21
6.2.2 提高服务水平	21
七、 模型评价与改进	21
7.1 模型的优点	21
7.2 模型的缺点	22
7.3 模型的改进	22
八、 参考文献	22
九、 附录	23

一、问题重述

1.1 问题背景

随着城市轨道交通的不断发展，客流量迅速增加且呈现出时空分布不均衡的特点。因此需要制定合理的列车时刻表，以发挥列车的运输能力，满足不同区段的客流需求，而在制定列车时刻优化表之前，需要先确定列车开行方案。

在城轨运营中，大小交路是城轨交通最常用的交路模式，适用于区段客流不均衡、客流断面突变明显的线路。有利于提高城市轨道交通的服务水平、降低运营成本。同时，在制定列车开行方案时，需要以最小的企业运营成本和最大的服务水平来满足客流的需求。因此，如何设计合理高效的大小交路开行方案是优化城市轨道交通列车时刻表的关键问题。

1.2 问题提出

在下列问题中，只需制定单向的列车时刻表即可。

1.2.1 问题一

在满足客流需求的条件下，以企业运营成本最小化和服务水平最大化为目标，制定列车开行方案。即确定大交路区间列车的开行数量，小交路的运行区间以及开行数量。

1.2.2 问题二

在问题一制定的列车开行方案下，同样以企业运营成本最小和服务水平最大化且尽量满足客流需求为目标，制定等间隔的平行运行图。

1.2.3 问题三

对于降低企业运营成本和提高服务水平，你们团队有哪些好的方法或建议？基于客流和车站数据，提供相应的量化分析支持。

二、问题分析

2.1 问题一的分析

问题一选取了单向线路作为研究对象，给定了大小交路模式的交路模式，希望我们给出该种交路模式下，既能满足客流需求，又能实现企业运营成本最小化和服务水平最大化的列车开行方案。所以本问目标函数应从企业运营成本和服务水平出发，建立多目

标规划模型并求解。在大小交路模式下，小交路区间既有大交路列车也有小交路列车，其他区间只有大交路列车，因此我们将 30 个站点分为小交路区间和其他区间进行讨论。

同时，列车在实际运行过程中，两车在同一区间追踪运行时，需保留一定的安全间隔。除此之外，列车开行还受到停站时间、客流需求等多方面的约束，所以本问在建立多目标规划模型时应充分考虑其约束条件。

2.2 问题二的分析

问题二需要在问题一制定的列车开行方案下，制定等间隔的平行运行图。在问题一中，我们已经给出了满足客流需求、企业运营成本最小化、服务水平最大化、发车间隔、追踪间隔等约束条件的列车开行方案。并确定了列车发车频率、列车开行数量和小交路区间，故本问只需求解出每个站点的停站时间，即可绘制出对应的等间隔的平行运行图。

2.3 问题三的分析

问题三需要我们基于客流和车站数据，提供量化分析支持，给出能够降低企业运营成本和提高服务水平的方法或建议。根据问题一建立的目标函数，我们可以得到企业运营成本和服务水平与列车最大满载率、列车开行数量、小交路区间、站点位置有关。因此，我们可以对以上因素进行灵敏度分析，根据得到的量化分析提出方法和建议。

同时，在问题一、二的求解中，我们使用的是统一的列车编组，但在实际运营中，列车可根据实际情况进行灵活编组。因此，我们还可以研究不同列车编组会对企业运营成本和服务水平带来的影响，从而提出相关建议和方法。

三、模型假设与符号说明

3.1 模型假设

- 1、乘客到达每个站点服从均匀分布，并选择第一趟到达的列车出发；
- 2、乘客只会选择最简单的乘车方式，全程只选择一种交路，即不考虑乘客的换乘问题；
- 3、列车停站时间取决于乘客数量和乘客上下车时间；
- 4、所有列车运行速率相同，且在各自交路区间均为站站停，不考虑跨站停车；
- 5、客流在一段时间内保持稳定。

3.2 主要符号说明

注：此为本文的主要符号说明，其他符号解释详见正文部分。

符号	说明
n	车站数量
N_c	上线车组数（即列车开行数量）
i	交路类型（ $i=1$ 为大交路， $i=2$ 为小交路）
S_1	大交路起点站
S_n	大交路终点站
S_a	小交路起点站
S_b	小交路终点站
f_i	编组中交路类型为 i 的发车频率
T	T 为运营时段时长（根据题目有 $T=1h$ ）
L	线路的总长度（即站点 S_1 到站点 S_n 的总距离）
L_i	一辆列车在类型为 i 的交路上行走的总路程
L_{SUM}	所有列车在大、小交路的行走路程之和
P_o	列车定员（根据附件有 $P_o=1860$ ）
P_2	小交路区间的 OD 客流总量
P_1	其他区间的 OD 客流总量
$P_c^{(k)}$	某一断面 P_k 的断面客流（站点 $S_k \rightarrow S_{k+1}$ 记为断面 k ）
$Q_u^{(k)}$	站点 S_k 的上车人数
$Q_d^{(k)}$	站点 S_k 的下车人数
$Q^{(k)}$	站点 S_k 的总流量
$T_s^{(k)}$	列车在站点 S_k 需要提供给乘客上下车的停站时间
$T_{on}^{(2)}$	在区间 $[S_a, S_b]$ 的所有乘客的在车时间之和
$T_{on}^{(1)}$	其他区间的所有乘客的在车时间之和
$T_w^{(2)}$	第IV类乘客的等待时间
$T_w^{(1)}$	其余类型乘客的等待时间
T_{track}	追踪时间间隔
T_{stop}	列车停站时间
b_i	编组中交路类型为 i 的编组辆数
$\eta_{\max}$	列车最大满载率

四、问题一的模型建立与求解

4.1 模型的建立

在大小交路模式下，乘客通常被分为6种类型，如图1所示，其中 $S_1 \sim S_n$ 为大交路区间， $S_a \sim S_b$ 为小交路区间。

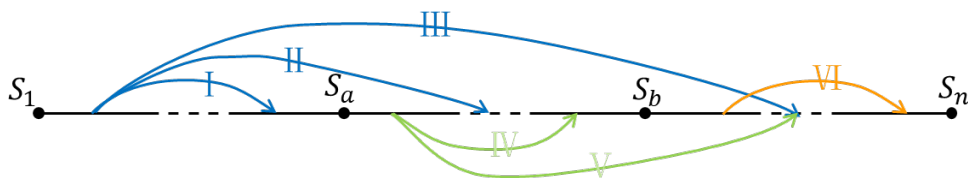


图 1 乘客类型分类示意图

在制定列车开行方案时，不仅需要满足客流需求，还需实现企业运营成本最小化和服务水平最大化，故本问的目标函数应从企业运营成本和服务水平出发，建立模型并求解。

4.1.1 企业运营成本

企业运营成本由固定成本（所需车辆的数量）和变动成本（列车总行走公里）两部分组成。

（1）固定成本（所需车辆的数量）

所需车辆为大交路与小交路所需车辆数量之和，故所需车辆数量 N_c 为：

$$N_c = \sum_{i=1}^2 f_i \cdot T \quad (1)$$

其中， f_i 为编组中交路类型为 i 的发车频率（ $i=1$ 为大交路， $i=2$ 为小交路）， T 为运营时段时长，根据题目有 $T=1h$ 。

（2）变动成本（列车行走总公里）

列车行走总公里为大交路行走总公里和小交路行走总公里之和，其中一辆列车在大交路行走的总路程 L_1 为：

$$L_1 = L \quad (2)$$

其中， L 为线路的总长度，即站点 S_1 到站点 S_n 的总距离。

一辆列车在小交路行走的总路程 L_2 为：

$$L_2 = \sum_{k=a}^{b-1} l_k \quad (3)$$

其中， l_k 为站点 S_k 到站点 S_{k+1} 的距离。

故列车行走总公里 L_{SUM} 为所有列车在大、小交路的行走路程之和，其表达式如下：

$$L_{SUM} = T \sum_{i=1}^2 f_i \cdot L_i \quad (4)$$

综上，企业运营成本最小化的目标函数表示为：

$$COST_1 = \min N_c \quad (5)$$

$$COST_2 = \min L_{SUM} \quad (6)$$

4.1.2 服务水平

服务水平包括乘客在车时间（乘客从上车到下车所经过的时间，包括列车区间运行时间和停站时间两部分组成）和乘客等待时间（即乘客在站台候车的等待时间）。

由图 1 我们可以知道，不同类型的乘客可选择的交路类型不尽相同，具体选择方式如下表：

表 1 乘客类型及交路选择方式

乘客类型	起点	终点	可选择的交路类型
I 类乘客	S_1 与 S_a 之间	S_1 与 S_a 之间	大交路
II 类乘客	S_1 与 S_a 之间	S_a 与 S_b 之间	大交路
III 类乘客	S_1 与 S_a 之间	S_b 与 S_n 之间	大交路
IV 类乘客	S_a 与 S_b 之间	S_a 与 S_b 之间	大/小交路
V 类乘客	S_a 与 S_b 之间	S_b 与 S_n 之间	大交路
VI 类乘客	S_b 与 S_n 之间	S_b 与 S_n 之间	大交路

从表 1 不难看出，只有 IV 类乘客才有机会选择小交路列车，则小交路区间的 OD 客流总量 P_2 为：

$$P_2 = \sum_{s=a}^{b-1} \sum_{e=s+1}^b P_{(s,e)} \quad (7)$$

其中， $P_{(s,e)}$ 表示站点 S_s 到站点 S_e 的 OD 客流，则可得到其他区间的 OD 客流总量 P_1 为：

$$P_1 = \sum_{s=1}^{n-1} \sum_{e=s+1}^n P_{(s,e)} - P_2 \quad (8)$$

同时有某一站点 k 的断面客流 $P_c^{(k)}$ ：

$$P_c^{(k)} = \sum_{s=1}^k \sum_{e=k+1}^n P_{(s,e)} \quad (k=1, 2, 3, \dots, n-1) \quad (9)$$

（1）乘客在车时间

乘客在车时间包括列车区间运行时间和停站时间两部分组成，考虑到开行单一交路和开行大小交路时所有列车速度均相等，列车区间运行时间不会发生变化，因此乘客在车时间只考虑列车停站时间。

假设在站点 S_k 的上车人数为 $Q_u^{(k)}$ ，下车人数为 $Q_d^{(k)}$ ，则有：

$$Q_u^{(k)} = \sum_{e=k+1}^n P_{(k,e)} \quad (10)$$

$$Q_d^{(k)} = \sum_{s=1}^{k-1} P_{(s,k)} \quad (11)$$

其中， $P_{(k,e)}$ 表示站点 S_k 到站点 S_e 的 OD 客流， $P_{(s,k)}$ 表示站点 S_s 到站点 S_k 的 OD 客流，即可得到站点 S_k 的总流量 $Q^{(k)}$ 为：

$$Q^{(k)} = Q_u^{(k)} + Q_d^{(k)} \quad (12)$$

所以列车在站点 S_k 需要提供给乘客上下车的停站时间 $T_s^{(k)}$ 为：

$$T_s^{(k)} = Q^{(k)} \cdot \bar{t} \quad (13)$$

其中， $\bar{t}$ 为乘客平均上下车时间（h/人），根据附件 5 “其他数据” 可以知道，

$$\bar{t} = \frac{0.04}{3600} \text{ (h/人)}。$$

因此，可以得到在区间 $[S_a, S_b]$ 的所有乘客的在车时间之和 $T_{on}^{(2)}$ 为：

$$T_{on}^{(2)} = \frac{\sum_{k=a}^b (T_s^{(k)} \cdot P_c^{(k)})}{(f_1 + f_2)} \quad (14)$$

同理，其他区间的所有乘客的在车时间之和 $T_{on}^{(1)}$ 为：

$$T_{on}^{(1)} = \frac{\sum_{k=1}^{a-1} (T_s^{(k)} \cdot P_c^{(k)}) + \sum_{k=b+1}^n (T_s^{(k)} \cdot P_c^{(k)})}{f_1} \quad (15)$$

（2）乘客等待时间

通过查阅相关资料可知，列车发车间隔会对乘客的等待时间造成影响。当车站列车发车间隔小于 10 分钟时，乘客到达站点服从均匀分布，乘客的平均等待时间为发车间隔时间的 $1/2^{[1]}$ 。对于第 IV 类乘客，该类乘客的等待时间 $T_w^{(2)}$ 为：

$$T_w^{(2)} = \frac{1}{2} \cdot \frac{1}{\sum_{i=1}^2 f_i} P_2 \quad (16)$$

其余类型乘客的等待时间 $T_w^{(1)}$ 为：

$$T_w^{(1)} = \frac{1}{2} \cdot \frac{1}{f_1} P_1 \quad (17)$$

综上，服务水平最大化的目标函数表示为：

$$COST3 = \min(T_{on}^{(1)} + T_{on}^{(2)} + T_w^{(1)} + T_w^{(2)}) \quad (18)$$

4.1.3 约束条件

由于列车在开行过程中，还应考虑发车间隔（频率）、停站时间、客流需求等要求，故还需对模型加设约束条件，通过附件 5 “其他数据”可以得到以下约束：

(1) 最小/最大发车频率约束

$$\frac{3600}{360} \leq f_1 \leq \frac{3600}{120} \quad (19)$$

$$\frac{3600}{360} \leq f_2 \leq \frac{3600}{120} \quad (20)$$

$$f_i \in Z^+ \quad (21)$$

$$f_2/f_1 \in Z^+ \text{ 或 } f_2/f_1 \in 1/Z^+ \quad (22)$$

(2) 停站时间约束

$$20 \leq T_{stop} \leq 120 \quad (23)$$

(3) 最小追踪间隔时间约束

$$T_{track} \geq 108 \quad (24)$$

(4) 运用列车车辆约束

列车开行数量需满足客流需求，则有

$$f_1 \cdot T > \frac{\max(P_c^{(k)})}{P_o} \quad (k \in [1, a-1] \cup [b, n-1]) \quad (25)$$

$$(f_1 + f_2) \cdot T > \frac{\max(P_c^{(k)})}{P_o} \quad (k \in [a, b-1]) \quad (26)$$

4.2 模型的求解算法

在该问中，我们建立了以企业运营成本最小化和服务水平最大化为目标函数的多目标规划模型，可用非支配遗传算法（NSGA）进行求解。相比于传统的多目标遗传算法，NSGA 算法在进行选择操作之前对个体进行了快速非支配排序，增大了优秀个体被保留的概率，可以得到更好的结果。但在实际应用中，NSGA 仍具有一定的缺点：

(1) 算法计算量大；

(2) 没有应用精英策略。未通过精英策略提高优秀个体的保留概率，程序的执行速度无法提高；

(3) 需要人为地指定共享半径，对于经验的要求非常高。

针对以上缺点，Deb 等人在 2002 年提出了 NSGA-II 算法^[2]，相比于 NSGA 算法，NSGA-II 算法做了如下改进：

(1) 使用快速非支配排序法，将计算复杂度由 $O(mN^3)$ 降到 $O(mN^2)$ ，大大减少了算法的计算时间；

(2) 应用精英策略，保证了优秀个体能够有更大的概率被保留；

(3) 用拥挤度的方法代替了需要人为地指定共享半径，保证了种群个体的多样性，有利于个体能够在整个区间及逆行选择、交叉和变异。

NSGA-II 算法无论在优化效果还是运算时间等方面都在优于 NSGA 算法，故针对问题一，我们选择 NSGA-II 算法进行求解。

4.2.1 NSGA-II 算法

(1) 快速非支配型排序

通过快速非支配型排序计算每个个体的非支配等级（Pareto 等级）。假设种群为 P ，当前 Pareto 最优解的个体的非支配等级为 1，除去这些等级为 1 的个体，组成新的种群 P' 。在新种群 P' 中最优解的非支配等级为 2，以此类推计算出种群 P 中的所有个体的非支配等级。设 n_p 为种群中支配个体的个数， S_p 为种群中被个体 p 支配的个体集合，则计算的具体步骤如下：

(i) 首先将种群中所有不被任何点支配的点（即 $n_p=0$ 的个体）挑选处理，保存在集合 F_1 中；

(ii) 对于集合 F_1 中的每个个体 i ，其所支配的个体集合为 S_i ，遍历 S_i 中的每个个体 l ，执行 $n_l = n_l - 1$ 删除该被支配点的支配关系，如果 $n_l = 0$ 则将个体 l 保存在集合 F_2 中。

(iii) 记 F_1 中得到的个体为第一个非支配层的个体，并以 F_2 作为当前集合，重复上述操作，直到整个种群被分级。需遍历整个种群，总计算复杂度是 $O(mN^2)$ 。

(2) 拥挤度

用于表现同一非支配等级个体之间的距离，为了保证种群个体的多样性，避免陷入局部最优解。用 d_i 表示种群中的给定点的周围个体密度，即第 i 个个体的拥挤距离，它指出了在个体 i 周围包含个体 i 本身但不包含其他个体的最小的长方形。

$$d_i = \sum_{m=1}^M |f_m^{i+1} - f_m^{i-1}| \quad (27)$$

其中 $i-1$ 与 $i+1$ 表示个体 i 沿着 i 所在的 Pareto Front Line 的两边邻近的两个个体。

f_m^{i+1} 与 f_m^{i-1} 分别表示 $i-1$ 与 $i+1$ 个个体第 m 个目标函数值 ($m=1, 2$)。

(3) 选择策略

首先比较非支配等级，等级小留下的可能性大，其次若非支配等级相同，则比较拥挤度。拥挤度大的留下的可能性大，若拥挤度相同，则随机留下一个。

(4) NSGA-II 算法流程

首先随机产生数量为 N 的初始种群，对其进行快速非支配型排序。而后与遗传算法类似，进行常规的选择、交叉、变异操作产生第一代子代种群。

从第二代开始，将父代优秀的个体与子代合并。首先对其进行快速非支配型排序，同时计算每个非支配层的个体进行拥挤度的计算；其次，根据非支配关系和拥挤度来选择合适的个体组成新的父代种群；最后再通过选择、交叉、变异产生子代，如此循环直到满足结束条件，NSGA-II 算法流程图如下：

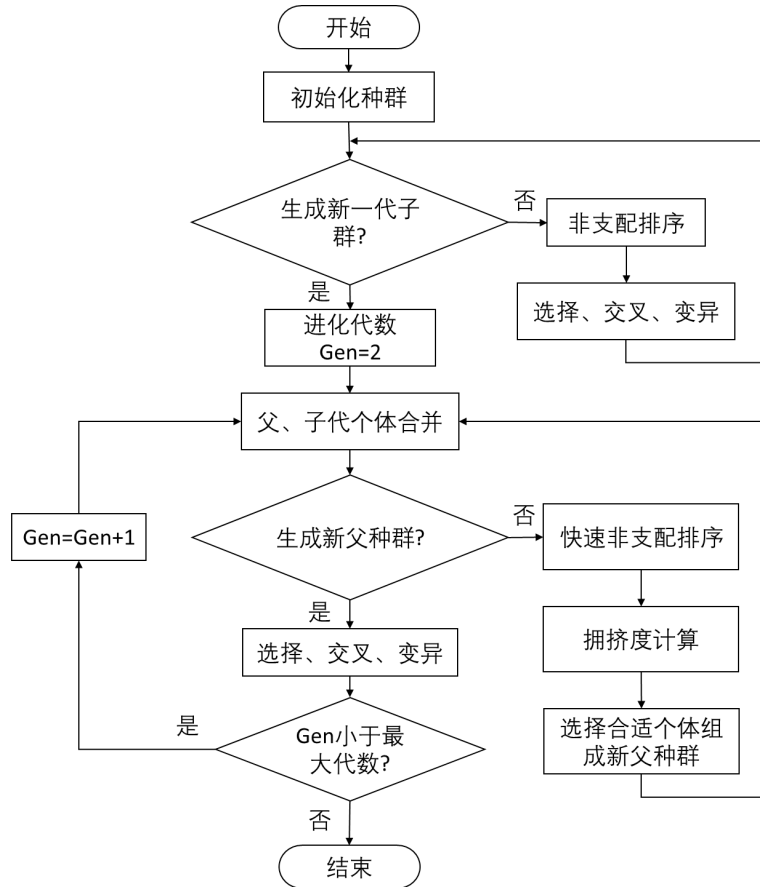


图 2 NSGA-II 算法流程图

4.2.2 模型的求解过程及结果

(1) 确定小交路区间起点站 S_a 和 S_b 的范围

在进行多目标优化之前，需要先确定小交路区间起点站 S_a 和 S_b 的范围。已知在小交路区间既有大交路列车，也有小交路列车，故我们取部分小交路区间 $[S_a, S_{a+2}]$ ，时段 $[t_1, t_5]$ 的列车发行情况作为研究对象（ $[t_1, t_5]$ 属于运营时段），如图 3 所示（蓝色为大交路列车发行情况，红色为小交路列车发行情况）：

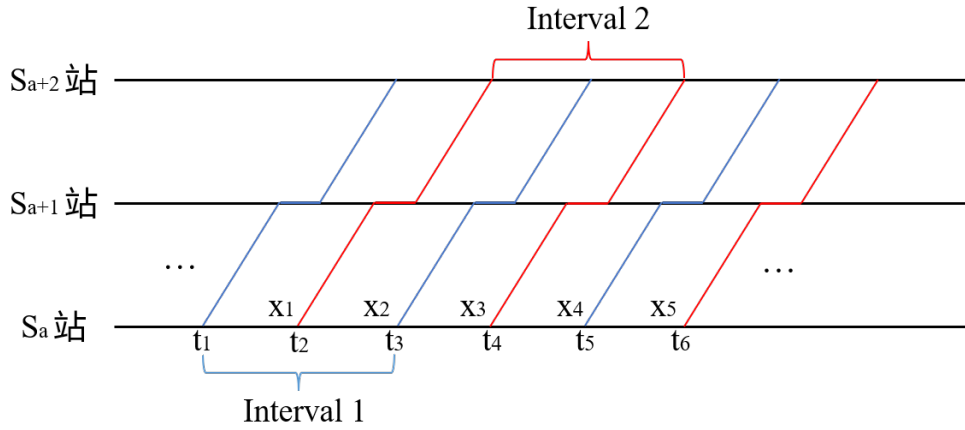


图 3 区间 $[S_a, S_{a+2}]$ ，时段 $[t_1, t_5]$ 的列车发行情况

根据最小追踪间隔时间约束可知， x_1, x_2, x_3, x_4 均需大于最小追踪间隔时间 108 秒，则可以得到该区间内大交路列车的发车间隔 $interval_1$ 和小交路列车的发车间隔 $interval_2$ 为：

$$interval_1 \geq 216 \quad (28)$$

$$interval_2 \geq 216 \quad (29)$$

由于其他区间内只含有大交路列车，根据图 3 可知，只要小区间内的大交路列车发车间隔满足最小追踪间隔约束，则其他区间内的大交路列车发车间隔也必然满足。故在该运行线路上，大交路列车和小交路列车的发车频率 f_1, f_2 应满足以下约束条件：

$$\frac{3600}{360} \leq f_1 \leq \frac{3600}{216} \quad (30)$$

$$\frac{3600}{360} \leq f_2 \leq \frac{3600}{120} \quad (31)$$

则可以得到交路列车和小交路列车的开行数量 N_{c1}, N_{c2} 应满足：

$$\frac{3600}{360} \leq N_{c1} = f_1 \cdot T \leq \frac{3600}{216} \quad (32)$$

$$\frac{3600}{360} \leq N_{c2} = f_2 \cdot T \leq \frac{3600}{216} \quad (33)$$

$$\frac{N_{c2}}{N_{c1}} \in Z^+ \text{ 或 } \frac{N_{c2}}{N_{c1}} \in \frac{1}{Z^+} \quad (34)$$

$$N_{c1}, N_{c2} \in Z^+ \quad (35)$$

得到 $N_{c1}, N_{c2} \in [10, 16]$ 且 $N_{c1} = N_{c2}$ 。

同时，对于开行数量 N_c 有：

$$N_c = \frac{\sum_{k=1}^{n-1} P_c^{(k)}}{P_o} \quad (36)$$

其中， P_o 为列车定员，根据附件 5 “其他数据” 可知 $P_o = 1860$ 。结合附件 4 “断面客流数据”，即可得到各个断面所需要的列车数量，具体数量如图 4：

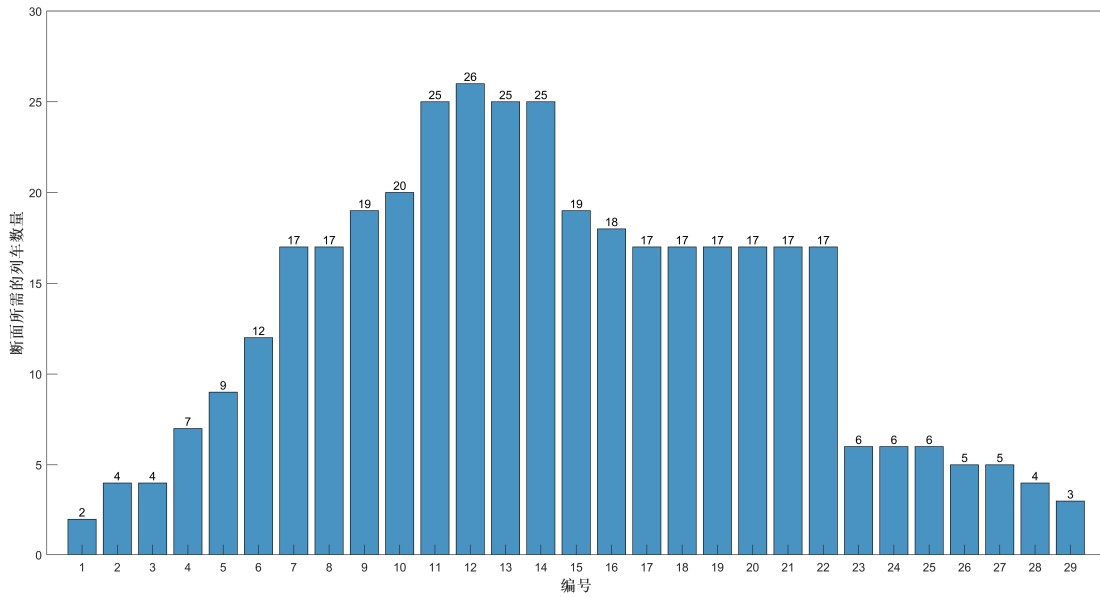


图 4 各个断面所需要的列车数量

从图中我们可以看出，在断面 7~断面 22，必须同时使用大、小交路列车才能满足客流需求，所以小交路起点站 $S_a \in [1, 7]$ ，小交路终点站 $S_b \in [22, 30]$ 。

(2) NSGA-II 求解

通过 Python 实现 NSGA-II 算法求解多目标优化问题。首先随机产生数量为 100 且

满足多目标规划模型约束条件的初始种群。而后根据非支配关系和拥挤度经过 50 次种群更新后，得到该问的最优解集，求解结果如图 5：

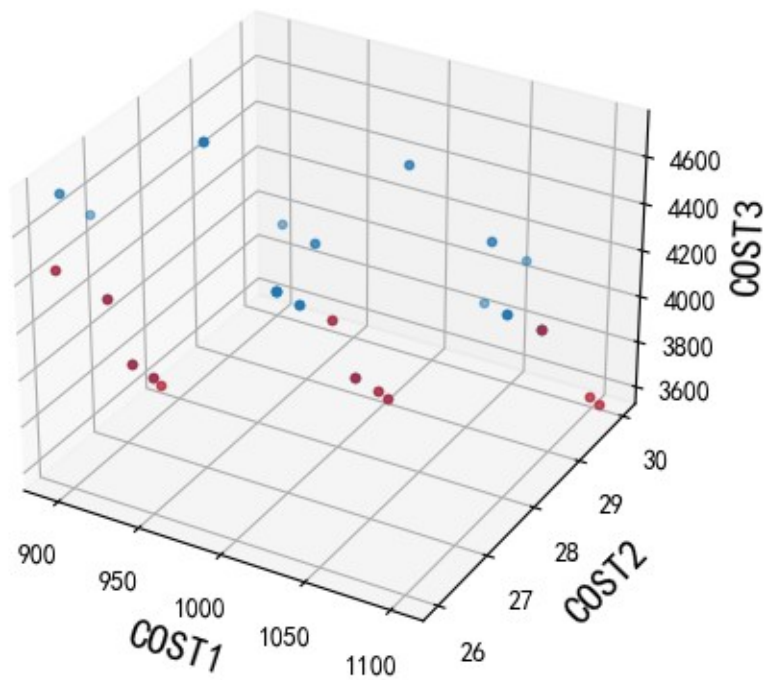


图 5 NSGA- II 算法求解结果

其中，红色点代表顶层解集（即最优解集中的更优解集），具体结果如表 2

具体结果如表 2:

表 2 NSGA- II 算法求解集合的顶层解集

序号	小交路 区间	大交路 列车数	小交路 列车数	<i>COST</i> 1 最优 目标函数值	<i>COST</i> 2 最优 目标函数值	<i>COST</i> 3 最优 目标函数值
1	[1,25]	14	14	1029.92	28	3803.43
2	[2,25]	13	13	938.418	26	4147.65
3	[2,25]	14	14	1010.6	28	3851.39
4	[2,25]	15	15	1082.79	30	3594.63
5	[2,26]	14	14	1024.97	28	3825.25
6	[2,26]	15	15	1098.18	30	3570.23
7	[5,22]	13	13	852.982	26	5040.11
8	[5,25]	13	13	893.802	26	4447.7
9	[5,25]	14	14	962.556	28	4130.01
10	[5,26]	13	13	907.14	26	4420.01
11	[5,26]	14	14	976.92	28	4104.3
12	[5,27]	13	13	924.69	26	4386.1
13	[5,27]	14	14	995.82	28	4072.81
14	[5,27]	15	15	1066.95	30	3801.29

为了从中选出最优的出行方案，我们采用 Topsis 方法计算出 14 组结果的得分，得分情况如表 3：

表 3 Topsis 法计算出 14 组结果的得分

得分排序	得分	对应结果序号
1	0.0849	8
2	0.0840	2
3	0.0829	10
4	0.0795	12
5	0.0766	9
6	0.0741	7
7	0.0726	11
8	0.0696	3
9	0.0675	13
10	0.0664	5
11	0.0657	1
12	0.0601	4
13	0.0584	6
14	0.0576	14

从表中我们可以看出结果 8 的得分最高，即结果 8 为最优开行方案，最终得到问题一的列车开行方案，具体方案如表 4：

表 4 问题一输出

	运行区间	运营里程	开行数量
大交路	$[S_1, S_{30}]$	522.184Km	13 列
小交路	$[S_5, S_{25}]$	371.618Km	13 列

五、问题二的求解

5.1 问题二求解过程

问题二需要在问题一制定的列车开行方案下，制定等间隔的平行运行图。在问题一中，我们给出的列车开行方案已满足以下条件：

- (1) 企业运营成本最小化；
- (2) 服务水平最大化；
- (3) 最小/大发车频率： $\frac{3600}{360} \leq f_i \leq \frac{3600}{120}$ ；
- (4) 追踪间隔时间： $T_{track} \geq 108$ ；
- (5) 满足客流需求。

同时，问题一已经确定了列车发车频率 f_i 、开行辆数 N_{c1}, N_{c2} 和小交路区间 $[S_a, S_b]$ 。此时，若要制定等间隔平行运行图，则还需确定每个站点的停站时间。问题一定义了列车在每个站点需要提供给乘客上下车的停站时间 $T_s^{(k)}$ ，则列车在每个站点的停站时间 T_{stop} 必须满足以下约束：

$$T_{stop} \geq T_s^{(k)} \quad (37)$$

$$20 \leq T_{stop} \leq 120 \quad (38)$$

根据附件 4 “断面客流数据”即可求出每个站点的停站时间，具体结果如图 6：

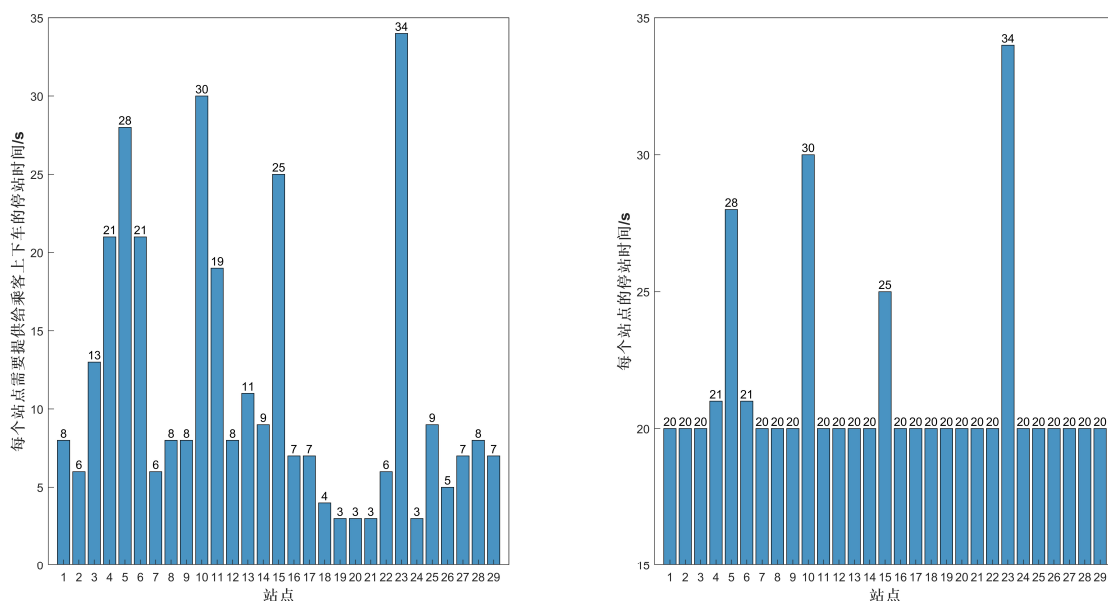


图 6 每个站点的停车时间

5.2 问题二求解结果

结合发车间隔，即可制定等间隔的平行运行图，本问使用了软件 qETRC 进行绘制，得到最终结果如图 7，具体输出数据见附件 7 “问题二输出”。

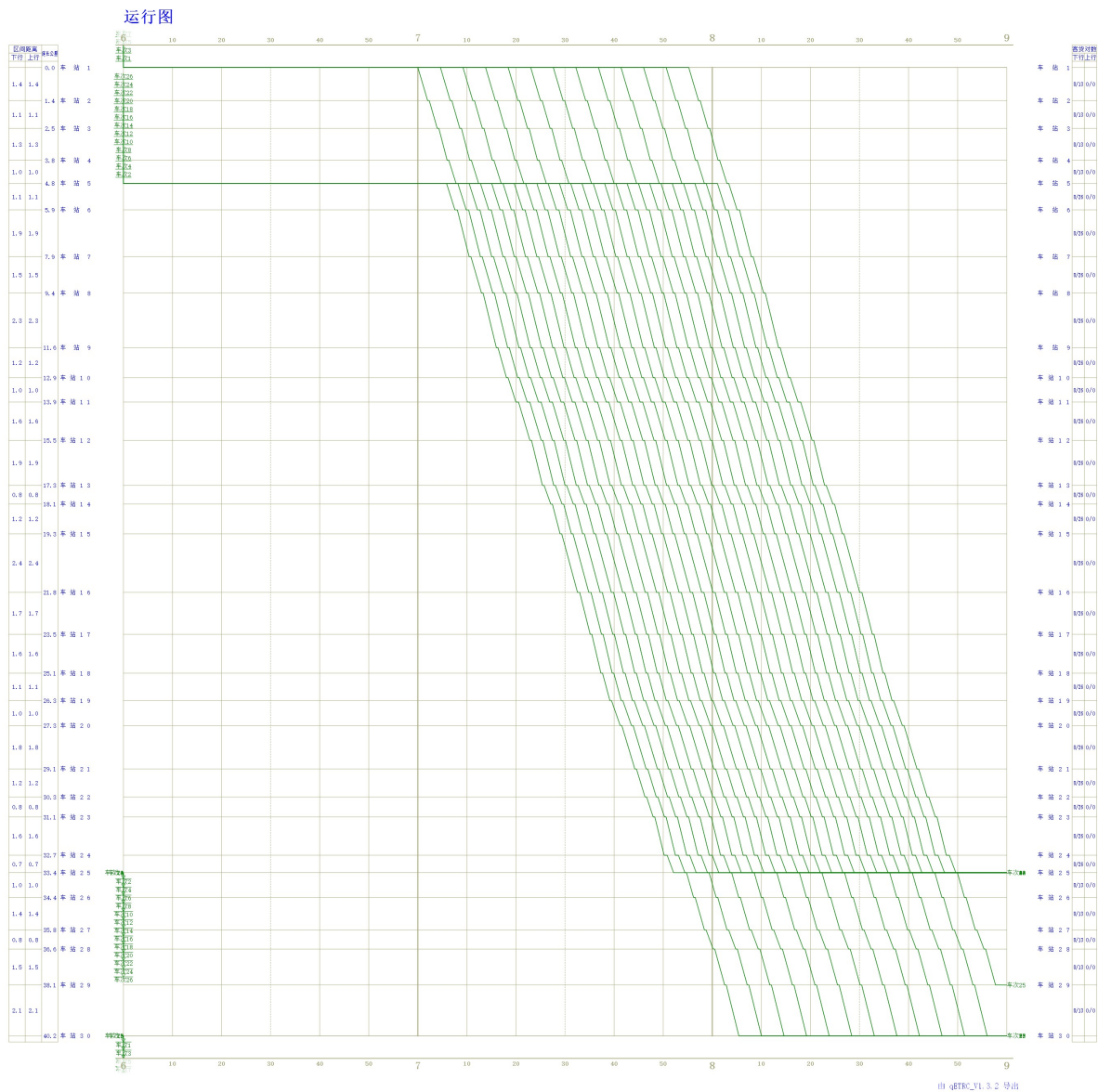


图 7 等间隔的平行运行图

六、问题三的求解

6.1 数据分析

6.1.1 站点位置分析

题目已经给定的可以作为起点/终点的站点如图 8（深色表示可以作为起点/终点，浅色表示未被选为起点/终点）：

站点1	站点2	站点3	站点4	站点5	站点6
站点7	站点8	站点9	站点10	站点11	站点12
站点13	站点14	站点15	站点16	站点17	站点18
站点19	站点20	站点21	站点22	站点23	站点24
站点25	站点26	站点27	站点28	站点29	站点30

图 8 站点作为交路起点/终点的情况

而列车在实际运营中，通常选取断面客流突变点作为交路起点/终点，根据图 9 我们可以得到该运行线路存在断面 7 ($S_7 \rightarrow S_8$)、断面 22 ($S_{22} \rightarrow S_{23}$)。

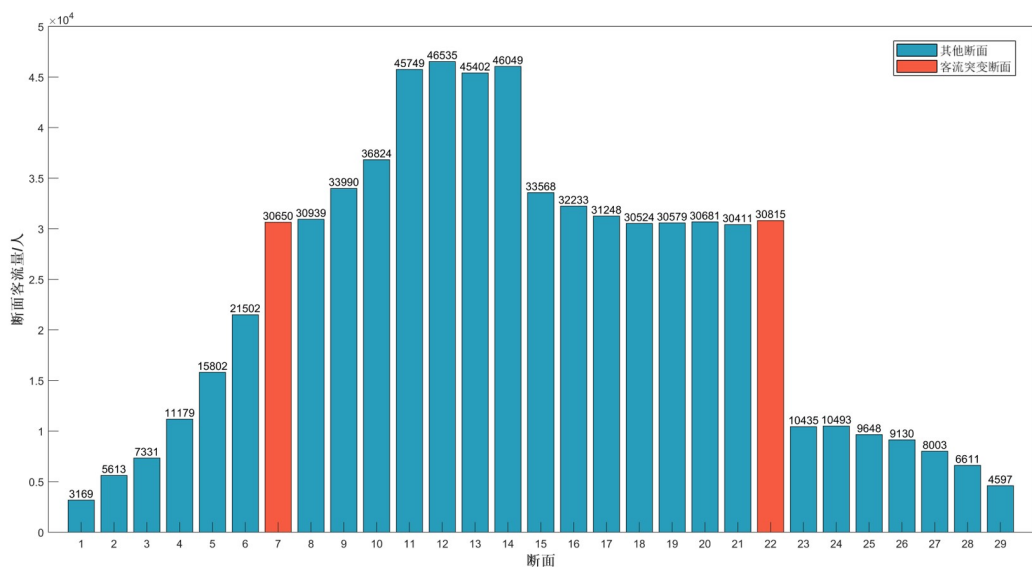


图 9 断面客流数据

但从图 8 我们可以看出，站点 S_7 和站点 S_{23} 并未被选作可以作为交路的起点/终点。故我们固定其他参数，令站点 S_7 和站点 S_{23} 也可以作为交路起点/终点，使用问题一建立的多目标规划模型，得到新的顶层解集，如表 6：

表 5 加入站点 S_7 、 S_{23} 作为交路起/终点后的顶层解集

序号	小交路 区间	大交路 列车数	小交路 列车数	$COST1$ 最优 目标函数值	$COST2$ 最优 目标函数值	$COST3$ 最优 目标函数值
1	[1,23]	13	13	926.289	26	4169.82
2	[1,23]	14	14	997.542	28	3871.98

3	[1,23]	15	15	1068.8	30	3613.85
4	[1,25]	15	15	1103.49	30	3549.86
5	[2,23]	13	13	908.349	26	4221.29
6	[2,23]	14	14	978.222	28	3919.77
7	[2,23]	15	15	1048.1	30	3658.45
8	[2,25]	13	13	938.418	26	4147.65
9	[2,25]	14	14	1010.6	28	3851.39
10	[2,25]	15	15	1082.79	30	3594.63
11	[2,26]	13	13	951.756	26	4119.5
12	[2,26]	14	14	1024.97	28	3825.25
13	[2,26]	15	15	1098.18	30	3570.23
14	[5,22]	13	13	852.982	26	5040.11
15	[5,23]	13	13	863.733	26	4520.11
16	[5,23]	15	15	996.615	30	3917.43
17	[5,25]	13	13	893.802	26	4447.7
18	[5,26]	13	13	907.14	26	4420.01
19	[5,26]	14	14	976.92	28	4104.3
20	[7,23]	13	13	824.2	26	5088.17

从表中结果可以看出，增设点站点 S_7 和站点 S_{23} 后，顶层解集中出现了许多以站点 S_{23} 为小交路区间终点的解。我们同样使用 Topsis 法计算每个结果的得分，具体得分如表 7:

表 6 新的顶层解集的得分情况

得分排序	得分	对应结果序号
1	0.0695	20
2	0.0673	14
3	0.0553	18
4	0.0549	17
5	0.0545	15
6	0.0501	19
7	0.0497	4
8	0.0495	13
9	0.0483	10
10	0.0471	3
11	0.0470	11
12	0.0466	12
13	0.0463	8
14	0.0457	5
15	0.0456	1
16	0.0454	16
17	0.0454	7
18	0.0453	9
19	0.0439	2

20	0.0425	6
----	--------	---

从表中我们可以看出，新得到的最优解为结果 14，即小交路区间为 $[S_7, S_{23}]$ ，大小交路列车发行数量均为 14 的开行方案。同问题一得到的最优开行方案相比，发现新的开行方案的 $COST1$ 最优目标函数值显著减小，所以若想降低企业运营成本，可以将站点 S_7 和站点 S_{23} 也作为交路的起点/终点。但此举会造成 $COST3$ 最优目标函数值的提高，即会降低服务水平，所以需要决策者结合实际要求再做出决策。

表 7 不同开行方案最优目标函数值对比

	$COST1$ 最优 目标函数值	$COST2$ 最优 目标函数值	$COST3$ 最优 目标函数值
新的开行方案	824.2	26	5088.17
问题一的开行方案	893.802	26	4447.7

6.1.2 满载率灵敏度分析

列车开行数量和列车发车频率都受列车最大满载率的直接影响，从而对企业运营水平和服务水平造成影响。通过分析不同满载率下的列车开行方案，可以得到满载率与企业运营水平和服务水平之间的关系。在问题一中，我们已经研究了最大满载率为 100% 的情况，现将最大满载率分别取 90%、95%、105%、110%、115%、120%的进行分析，灵敏度分级结果见表 9。

表 8 列车最大满载率的灵敏度分析结果

最大满载率	小交路 区间	大交路 列车数	小交路 列车数	$COST1$ 最优 目标函数值	$COST2$ 最优 目标函数值	$COST3$ 最优 目标函数值
90%	[5,25]	15	15	1082.79	30	3594.63
95%	[5,25]	14	14	962.556	28	4130.01
100%	[5,25]	13	13	893.802	26	4447.7
105%	[5,25]	12	12	825.048	24	4818.35
110%	[5,25]	12	12	825.048	24	4818.35
115%	[5,25]	11	11	756.294	30	5256.38
120%	[5,25]	11	11	756.294	30	5256.38

从表 9 中可以发现，当列车满载率越大时， $COST1$ 、 $COST2$ 最优目标函数值越小，成反比关系；而 $COST3$ 最优目标函数值越大，成正比关系。表明随着满载率的增加，企业运营成本会减小，但服务水平会降低。因此，当企业成本投资有限时，可通过增大满载率减少企业运营成本，但此举会造成旅客乘车体验欠佳，故需要决策者结合实际要求再做出决策。

6.1.3 列车开行数量灵敏度分析

列车开行数量会直接影响乘客的服务水平，列车开行数量越少，拥挤度也越大，反之亦然。因此，我们取同一区间 $[S_5, S_{25}]$ ，且满足约束条件的不同发行数量的大小列车进行分析，灵敏度分析结果见表 10。

表 9 列车开行数量的灵敏度分析结果

序号	大交路 列车数	小交路列 车数	<i>COST</i> 1 最优目 标函数值	<i>COST</i> 2 最优目 标函数值	<i>COST</i> 3 最优目 标函数值
1	13	13	893.802	26	4447.7
2	14	14	962.556	28	4130.01
3	15	15	1031.31	30	3854.68

从表 10 中我们可以发现，在相同区间且满足约束条件的情况下，列车开行数量越多，*COST*1、*COST*2 最优目标函数值越大，成正比关系；而*COST*3 最优目标函数值越小，成反比关系。表明在可行范围之内增加列车开行数量，可以提高服务水平，但会带来企业运营成本的提高。因此，企业在资金允许的情况下，可以适当提高列车开行数量来提高服务水平，从而吸引更多的乘客。

6.1.4 小交路区间灵敏度分析

小交路区间的长短会影响列车的折返次数，从而影响企业运营成本，为了进一步得到小交路区间对企业运营成本和服务水平带来的影响，我们在问题一给出的顶层解集中选取了不同小交路区间、相同列车开行数量的结果，对其最优目标函数值进行比较，对比结果见表 11。

表 10 不同小交路区间最优目标函数值对比

序号	小交路 区间	大交路 列车数	小交路 列车数	<i>COST</i> 1 最优 目标函数值	<i>COST</i> 2 最优 目标函数值	<i>COST</i> 3 最优 目标函数值
1	[5,22]	13	13	852.982	26	5040.11
2	[5,25]	13	13	893.802	26	4447.7
3	[5,26]	13	13	907.14	26	4420.01
4	[5,27]	13	13	924.69	26	4386.1

从表中我们可以看出，随着小交路区间增大，*COST*1 最优目标函数值也增大，而*COST*3 最优目标函数值则减小，表明小交路区间越大，企业运营成本会越大，但服务水平会越高。故企业可以通过减小小交路区间来降低企业运营成本，也可以通过增大小交路区间来提高服务水平。

6.1.5 车辆灵活编组分析

本文在研究车辆开行方案时，大小交路列车采用了统一的编组方案（即统一的编组数量），但在实际过程中，可根据实际情况进行灵活编组。通过改变大小交路列车的编组数量，对比各种编组量数下的最优目标函数值，可获得最优方案。通过查阅相关资料可知，一节列车通常可载 310 个乘客^[3]，我们在模型一的基础上，加入列车编组数量作为决策变量，则原始模型中的列车数量 N_c 和列车行走总公里 L_{SUM} 变为：

$$N_c = \sum_{i=1}^2 f_i \cdot T \cdot b_i \quad (39)$$

$$L_{SUM} = T \sum_{i=1}^2 f_i \cdot L_i \cdot b_i \quad (40)$$

其中 b_i 表示编组中交路类型为 i 的编组辆数。

为了避免改变列车编组造成不满足客流需求的情况，我们在小交路区间和其他区间分别增设以下约束：

$$\frac{P_c^{(k)}}{P_o \cdot \sum_{i=1}^2 b_i \cdot f_i} \leq \eta_{\max} \quad (41)$$

$$\frac{P_c^{(k)}}{P_o \cdot b_1 \cdot f_1} \leq \eta_{\max} \quad (42)$$

其中， $\eta_{\max}$ 为最大载客率。同样基于企业运营成本最小化目标函数 $COST1$ 、 $COST2$ 和服务水平最大化目标函数 $COST3$ 建立多目标规划模型，利用 NSGA-II 算法对模型进行求解。

我们在 NSGA-II 得到的顶层解集选取了小交路区间均为 $[S_5, S_{25}]$ ，大小交路列车开行数量均为 13 的结果，对其 $COST1$ 最优目标函数值进行比较，比较结果见表 12。

表 11 不同编组下的 $COST1$ 最优目标函数值比较

序号	大交路列车编组数量	小交路列车编组数量	$COST1$ 最优目标函数值
1	4	8	5061.68
2	5	7	5212.25
3	6	6	5362.81
4	7	5	5513.38
5	8	4	5563.94
6	9	3	5814.51

从表中我们可以看出，当小交路区间和列车发行数量一定时，大交路列车编组数量

越大（小交路列车编组数量越小）， $COST1$ 最优目标函数值越大，即企业运营成本越高。所以若想降低企业运营成本，可以适当改变大小交路列车的编组数量。

6.2 方法和建议

根据“6.1 数据分析”，我们得到站点位置、满载率、列车开行数量、小交路区间和列车编组与企业运营成本和服务水平之间的量化关系，据此我们提出以下建议，用于降低企业运营成本或提高服务水平。

6.2.1 降低企业运营成本

- （1）加入站点 S_7 、 S_{23} 作为交路起/终点；
- （2）提高列车最大满载率；
- （3）减少列车开行数量；
- （4）减小小交路区间；
- （5）采用列车灵活编组，且大交路列车编组数量小于小交路列车编组数量。

6.2.2 提高服务水平

- （1）减小列车最大满载率；
- （2）增加列车开行数量；
- （3）增大小交路区间；
- （4）采用列车灵活编组，且小交路列车编组数量大于小交路列车编组数量。

总体来说，企业运营成本的降低必然会带来服务水平的降低，故在实际情况中，往往需要决策者根据实际要求，有所偏重地做出决策。

七、模型评价与改进

7.1 模型的优点

（1）本文以企业运营成本最小化和服务水平最大化为目的建立的多目标优化模型，从列车整体运行水平及区间运行水平，分别进行约束，以满足客流需求。

（2）本文使用的基于 NSGA-II 算法的多目标优化模型采用快速非支配排序，和拥挤度比较算子，求解可得多个帕累托最优解，相较于加权系数等单目标优化，更具实用价值，可以为轨道交通提供多种解决方案。

（3）问题三中，从车站选取、列车满载率、列车开行数量、小交路区间等多角度

提出建议。并结合实际情况，探讨了列车灵活编组对企业运营成本和服务水平带来的影响，让提出的建议更具实际应用价值。

7.2 模型的缺点

（1）本文模型并未考虑乘客的换乘情况，但在实际情况中，对于起始站点位于小交路区间，终点站点位于大交路区间的乘客，还可以选择换乘的方式。故得出的最优列车开行方案与实际情况存在一定差距。

（2）对于问题一的多目标优化模型仅选择了一种启发式算法求解，未与其他算法进行对比分析。

（3）本文问题求解时考虑的列车停站方案为“站站停”的停站方案，但实际应用中为提高列车区间通过能力，可采取开行大站快运列车，客流小站不停靠的策略。

7.3 模型的改进

（1）增加对换乘情况的考虑；

（2）研究“大站快运列车，客流小站不停靠”策略下的最优列车开行方案，并与“站站停”方式进行对比。

（3）建模时加入票价等影响因子。

八、参考文献

- [1] 李星阳. 城市轨道交通线路列车多编组方案研究[D].北京交通大学,2018.
- [2] Deb K, Pratap A, Agarwal S, et al. A fast and elitist multiobjective genetic algorithm: NSGA-II[J]. IEEE transactions on evolutionary computation, 2002, 6(2): 182-197.
- [3] 陈龙. 基于灵活编组的城市轨道交通大小交路列车开行方案研究[D].兰州交通大学,2021.
- [4] 安志龙,马丽,崔虎,安志学.基于遗传算法的城市轨道交通大小交路模式列车开行方案研究[J].河南科技,2022,41(18):6-10.
- [5] 王慧. 城市轨道交通大小交路开行方案优化研究[D].北京交通大学,2020.
- [6] 沈强. 城市轨道交通大小交路开行方案设计研究[D].兰州交通大学,2020.

九、附录

附录 1

介绍：该代码由 Python 编写，为问题一的多目标规划模型。

```
import pandas as pd
import numpy as np

class Target:
    def __init__(self,Sa,Sb,f1,f2):
        self.Sa = Sa
        self.Sb = Sb
        self.f1 = f1
        self.f2 = f2
        self.N = 30
        self.L1 = 0
        self.L2 = 0
        self.Q1 = 0
        self.Q2 = 0
        self.Quv = []
        self.Dm = []
        self.tau = 0.04 # 乘客平均上下车时间

    def build(self):
        # 断面客流
        fpath1 = r"./Data_B/附件 4：断面客流数据.xlsx"
        Dm = pd.read_excel(fpath1)
        Dm = np.array([0] + list(Dm.iloc[:, 1]) + [0])
        self.Dm = Dm

        # 区间客流
```

```

fpath2 = r"./Data_B/附件 3： OD 客流数据.xlsx"
Quv = pd.read_excel(fpath2)
Quv = np.array(Quv)
self.Quv = Quv
Q1, Q2 = 0, 0  # Q2 IV 区间流量  Q1 其他区间
for i in range(self.Sa, self.Sb):
    for j in range(i + 1, self.Sb + 1):
        Q2 += Quv[i, j]

for i in range(1, self.N):
    for j in range(i + 1, self.N + 1):
        Q1 += Quv[i, j]
Q1 -= Q2

self.Q1 = Q1
self.Q2 = Q2

# 区间距离
fpath3 = r"./Data_B/附件 2： 区间运行时间.xlsx"
L = pd.read_excel(fpath3)
Ld = np.array([0] + list(L.iloc[:, 2]))
L1 = sum(Ld)
L2 = 0
for i in range(self.Sa, self.Sb):
    L2 += Ld[i]
self.L1 = L1
self.L2 = L2

def Z1(self):
    Vm = self.f1 * self.L1 + self.f2 * self.L2

```

```

return -Vm

def Z2(self):
    n = self.f1 + self.f2
    return -n

def fuc_tm(self,m):
    dm1 = 0
    dm2 = 0
    for i in range(m + 1, self.N + 1):
        dm1 += self.Quv[m][i]
    for j in range(1, m):
        dm2 += self.Quv[j][m]
    tm = (dm2 + dm1) * self.tau / 3600
    return tm

def Z3(self):
    w1 = self.Q2 / (2 * (self.f1 + self.f2)) # IV 客流等待时间
    w2 = self.Q1 / (2 * self.f1) # 非 IV 客流等待时间
    w31, w32 = 0, 0 # 在车时间
    for i in range(self.Sa, self.Sb + 1):
        w32 += self.fuc_tm(i) * self.Dm[i]
    for j in range(1, self.N + 1):
        w31 += self.fuc_tm(j) * self.Dm[j]

    w31 = (w31 - w32) / self.f1
    w32 = w32 / (self.f1 + self.f2)

    return -(w1 + w2 + w31 + w32)

```

附录 2

介绍：该代码由 Python 编写，为 NSGA-II 算法，用于求解多目标规划模型。

```
import math
import random
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from model_v2 import *
import warnings
warnings.filterwarnings("ignore")
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

def index_of(a, list):
    for i in range(0, len(list)):
        if list[i] == a:
            return i
    return -1

def sort_by_values(list1, values):
    sorted_list = []
    while len(sorted_list) != len(list1):
        if index_of(min(values), values) in list1:
            sorted_list.append(index_of(min(values), values))
            values[index_of(min(values), values)] = math.inf
            #          infinted
    return sorted_list
```

```

def fast_non_dominated_sort(values1, values2, values3):
    S = [[] for i in range(0, len(values1))]
    front = [[]]
    n = [0 for i in range(0, len(values1))]
    rank = [0 for i in range(0, len(values1))]
    # 评级
    for p in range(0, len(values1)):
        S[p] = []
        n[p] = 0
        for q in range(0, len(values1)):
            # step2:p > q 即如果 p 支配 q,则
            if (values1[p] >= values1[q] and values2[p] >= values2[q] and
values3[p] >= values3[q])\
                and not all([values1[p]==values1[q],values2[p] ==
values2[q],values3[p] == values3[q]]):

                if q not in S[p]:
                    # 同时如果 q 不属于 sp 将其添加到 sp 中
                    S[p].append(q)
                # 如果 q 支配 p
                elif (values1[q] >= values1[p] and values2[q] >= values2[p] and
values3[q] >= values3[p])\
                    and not all([values1[q]==values1[p],values2[q] ==
values2[p],values3[q] == values3[p]]):
                    # 则将 np+1
                    n[p] = n[p] + 1
        if n[p] == 0:
            # 找出种群中 np=0 的个体
            rank[p] = 0
            # 将其从 pt 中移去

```

```

        if p not in front[0]:
            # 如果 p 不在第 0 层中
            # 将其追加到第 0 层中
            front[0].append(p)

i = 0
while (front[i] != []):
    # 如果分层集合不为空,
    Q = []
    for p in front[i]:
        for q in S[p]:
            n[q] = n[q] - 1
            # 则将 fk 中所有给对应的个体 np-1
            if (n[q] == 0):
                # 如果 nq==0
                rank[q] = i + 1

                if q not in Q:
                    Q.append(q)

    i = i + 1
    # 并且 k+1
    front.append(Q)

del front[len(front) - 1]

return front
# 返回将所有个体分层后的结果

def crowding_distance(values1, values2, values3, front):
    distance = [0 for _ in range(0, len(front))]

```

```

# 初始化个体间的拥挤距离

sorted1 = sort_by_values(front, values1[:])
sorted2 = sort_by_values(front, values2[:])
sorted3 = sort_by_values(front, values3[:])

# 基于目标函数 1 和目标函数 2 对已经划分好层级的种群排序
distance[0] = 444444444444444444
distance[len(front) - 1] = 444444444444444444

for k in range(1, len(front) - 1):
    distance[k] = distance[k] + (values1[sorted1[k + 1]] - values1[sorted1[k - 1]]) /
(max(values1) - min(values1))
    distance[k] = distance[k] + (values2[sorted2[k + 1]] - values2[sorted2[k - 1]]) /
(max(values2) - min(values2))
    distance[k] = distance[k] + (values3[sorted3[k + 1]] - values3[sorted3[k - 1]]) /
(max(values3) - min(values3))
return distance

def crossover(a, b, min, max):
    r = random.random()
    if r > 0.5:
        return mutation((a + b) / 2, min, max)
    else:
        return mutation((a - b) / 2, min, max)

# 函数进行变异操作
def mutation(solution, min, max):
    mutation_prob = random.random()
    if mutation_prob < 1:
        solution = min + (max - min) * random.random()
    return solution

```



```

def panduan(sa,sb,a,b):
    flag1 = False
    flag2 = False
    ss = [1,2,5,8,10,14,17,18,21,22,25,26,27,30]
    a_b = sb-sa
    if (sa in ss) and (sb in ss) and 3 <= a_b <= 24:
        flag1 = True
    if (25<a+b<=30) and (a/b % 1 == 0 or b/a % 1 == 0):
        flag2 = True

    return all([flag1,flag2])

def randm():
    solution_a = list()
    solution_b = list()
    solution_sa = list()
    solution_sb = list()
    for i in range(0, pop_size):
        while True:
            x1 = round(min_fk + (max_fk - min_fk) * random.random())
            x2 = round(min_fk + (max_fk - min_fk) * random.random())
            s1 = round(sa_min + (sa_max - sa_min) * random.random())
            s2 = round(sb_min + (sb_max - sb_min) * random.random())
            if panduan(s1,s2,x1,x2):
                solution_a.append(x1)
                solution_b.append(x2)
                solution_sa.append(s1)
                solution_sb.append(s2)
                break
    return solution_a,solution_b,solution_sa,solution_sb

```

```

if __name__ == '__main__':

    pop_size = 60
    max_gen = 10
    # 迭代次数
    # Initialization
    min_fk = 10
    max_fk = 16
    sa_min = 1
    sa_max = 6
    sb_min = 22
    sb_max = 30

    solution_a, solution_b, solution_sa, solution_sb = randm()
    # 随机生成变量
    gen_no = 0
    while (gen_no < max_gen):
        function1_values = []
        function2_values = []
        function3_values = []
        ob_list = []
        for i in range(0, pop_size):
            ob = Target(solution_sa[i], solution_sb[i], solution_a[i], solution_b[i])
            ob.build()
            function1_values.append(ob.Z1())
            function2_values.append(ob.Z2())
            function3_values.append(ob.Z3())
            ob_list.append(ob)
        # 生成两个函数值列表，构成一个种群
        non_dominated_sorted_solution = \

```

```

        fast_non_dominated_sort(function1_values[:,],        function2_values[:,],
function3_values[:,])
    # 种群之间进行快速非支配性排序,得到非支配性排序集合

    tt = list()

    for valuez in non_dominated_sorted_solution[0]:
        ob1 = ob_list[valuez]

        t                =                [solution_sa[valuez],
solution_sb[valuez],solution_a[valuez],solution_b[valuez],ob1.Z1(), ob1.Z2(), ob1.Z3())

        print(t, end=" ")

        tt.append(t)

    print("\n")

    cc = np.array(tt)

    crowding_distance_values = []

    # 计算非支配集合中每个个体的拥挤度
    for i in range(0, len(non_dominated_sorted_solution)):

        crowding_distance_values.append(

            crowding_distance(function1_values[:,], function2_values[:,],

                                function3_values[:,],

non_dominated_sorted_solution[i][:])

        )

        solution2_a = solution_a[:]
        solution2_b = solution_b[:]
        solution2_sa = solution_sa[:]
        solution2_sb = solution_sb[:]

        while (len(solution2_a) != 2 * pop_size):

            a1 = random.randint(0, pop_size - 1)

            b1 = random.randint(0, pop_size - 1)

            # 选择

            aa = round(crossover(solution_a[a1], solution_a[b1],min_fk,max_fk))

```

```

bb = round(crossover(solution_b[a1], solution_b[b1],min_fk,max_fk))
saa = round(crossover(solution_sa[a1], solution_sa[b1],sa_min,sa_max))
sbb = round(crossover(solution_sb[a1], solution_sb[b1],sb_min,sb_max))
if panduan(saa, sbb, aa, bb):
    solution2_a.append(aa)
    solution2_b.append(bb)
    solution2_sa.append(saa)
    solution2_sb.append(sbb)
# 随机选择，将种群中的个体进行交配，得到子代种群 2*pop_size

function1_values2 = []
function2_values2 = []
function3_values2 = []
for i in range(0, pop_size):
    ob2 = Target(solution2_sa[i], solution2_sb[i], solution2_a[i],
solution2_b[i])
    ob2.build()

    function1_values2.append(ob2.Z1())
    function2_values2.append(ob2.Z2())
    function3_values2.append(ob2.Z3())

non_dominated_sorted_solution2 = \
    fast_non_dominated_sort(function1_values2[:,], function2_values2[:,],
function3_values2[:,])
# 将两个目标函数得到的两个种群值 value,再进行排序 得到 2*pop_size 解

crowding_distance_values2 = []
for i in range(0, len(non_dominated_sorted_solution2)):
    crowding_distance_values2.append(

```

```

        crowding_distance(function1_values2[:, function2_values2[:,
                                function3_values2[:,
non_dominated_sorted_solution2[i][:]))
    # 计算子代的个体间的距离值
    new_solution = []
    for i in range(0, len(non_dominated_sorted_solution2)):
        non_dominated_sorted_solution2_1 = [
            index_of(non_dominated_sorted_solution2[i][j],
non_dominated_sorted_solution2[i]) for j in
                range(0, len(non_dominated_sorted_solution2[i]))]
        # 排序
        front22 = sort_by_values(non_dominated_sorted_solution2_1[:,
crowding_distance_values2[i][:])
        front = [non_dominated_sorted_solution2[i][front22[j]] for j in
                range(0, len(non_dominated_sorted_solution2[i]))]
        front.reverse() # 反转列表
        for value in front:
            new_solution.append(value)
            if (len(new_solution) == pop_size):
                break
        if (len(new_solution) == pop_size):
            break

    solution_a = [solution2_a[i] for i in new_solution]
    solution_b = [solution2_b[i] for i in new_solution]
    solution_sa = [solution2_sa[i] for i in new_solution]
    solution_sb = [solution2_sb[i] for i in new_solution]
    gen_no = gen_no + 1

fig = plt.figure()
ax = fig.gca(projection='3d')

```

```
function1 = [-i for i in function1_values]
function2 = [-i for i in function2_values]
function3 = [-j for j in function3_values]
ax.set_xlabel('COST1', fontsize=15)
ax.set_ylabel('COST2', fontsize=15)
ax.set_zlabel('COST3', fontsize=15)
ax.plot(function1, function2, function3, '.', alpha=0.5)

a = np.unique(cc, axis=0) # 顶层去重
function11 = [-a[i][4] for i in range(len(a))]
function22 = [-a[i][5] for i in range(len(a))]
function33 = [-a[i][6] for i in range(len(a))]
ax.plot(function11, function22, function33, 'r.', alpha=0.5)
dd = pd.DataFrame(a)
dd.to_excel(r"./结果数据表/顶层最优解 2.xlsx")
plt.show()
```